

Software Requirements Specification (SRS)

Project: AI-Powered KYC – "Reimagining KYC with AI"

Prepared for: Round 2 Hackathon Submission

Prepared by: Codeists

Date: 13/11/2025

1. Introduction

1.1 Purpose

This SRS specifies functional and non-functional requirements for an AI-powered KYC onboarding system that automates document collection, identity verification, risk scoring, and provides auditability and explainability for compliance.

1.2 Scope

The system supports end-to-end KYC onboarding: document upload, OCR extraction, selfie-ID face verification, liveness detection, entity validation, risk scoring, explainability, immutable audit logging, and decisioning (auto-approve / manual review). It targets hackathon deployment (single environment) with a container-ready architecture for future scaling.

1.3 Definitions, Acronyms, Abbreviations

- API: Application Programming Interface
 - OCR: Optical Character Recognition
 - PII: Personally Identifiable Information
 - SHA / SHAP: SHapley Additive exPlanations
 - pgvector: PostgreSQL extension for vector embeddings
-

2. Overall Description

2.1 Product Perspective

Modular, asynchronous backend (FastAPI) with a React + Vite + TailwindCSS frontend. PostgreSQL (with pgvector) is the persistent store. All components operate as services orchestrated by the backend.

2.2 User Classes and Characteristics

- **End users (customers):** upload documents and selfie; receive guided feedback and final decision.
- **Compliance reviewers:** view explainability outputs, audit logs, and manually review flagged cases.
- **System administrators / DevOps:** deploy and maintain application.

2.3 Operating Environment

- Backend: Python + FastAPI (async) running on Linux (single environment for hackathon).
- Frontend: Browser (modern).
- Database: PostgreSQL with pgvector.
- Transport: TLS 1.3.
- Storage: Encrypted at rest (AES-256).

2.4 Design & Implementation Constraints

- Use only the components listed in Round 1 (EasyOCR, Tesseract, FaceNet, Mediapipe, BERT, XGBoost, SHAP, PostgreSQL + pgvector, FastAPI, React/Vite/TailwindCSS).
- All PII must be encrypted at rest with AES-256 and in transit with TLS 1.3.
- PostgreSQL must maintain immutable audit logs.

2.5 Assumptions & Dependencies

- Users have device cameras for selfie & simple document capture.
 - Demo and code links will be public for submission.
 - Infrastructure for the hackathon is a single environment; containerization is optional but supported.
-

3. Functional Requirements (FR)

FR-1: User Onboarding

- FR-1.1: The system shall allow a user to start a KYC application and upload required documents (ID, address proof, utility bill) and a selfie.
- FR-1.2: The system shall provide live frontend feedback (e.g., "Retake – glare detected") during document capture.

FR-2: OCR & Text Extraction

- FR-2.1: The system shall extract text from uploaded documents using EasyOCR as primary OCR.
- FR-2.2: The system shall fallback to Tesseract when document layout is cleaner or EasyOCR confidence is below threshold.
- FR-2.3: The system shall store extracted text and OCR confidence values.

FR-3: Face Verification & Liveness

- FR-3.1: The system shall perform one-to-one facial similarity between selfie and ID photo using FaceNet embeddings.
- FR-3.2: The system shall run Mediapipe-based liveness detection (blink/motion cues) and store a liveness confidence value.

FR-4: Entity Validation

- FR-4.1: The system shall process OCR output through BERT to extract and standardize entities: Name, Date of Birth, Address.
- FR-4.2: The system shall correct obvious OCR inconsistencies and persist validated fields.

FR-5: Risk Scoring & Decisioning

- FR-5.1: The system shall compute a dynamic risk score using XGBoost combining features: document quality, liveness confidence, location anomalies, behavioral patterns.
- FR-5.2: The system shall auto-approve low-risk applications and flag medium/high-risk applications for manual review.
- FR-5.3: The system shall store the full feature map used for each risk score.

FR-6: Explainability & Audit

- FR-6.1: The system shall generate SHAP explanations for each risk decision and store them.
- FR-6.2: The system shall store immutable audit logs for every major action (OCR extraction, face match, liveness result, risk calculated, decision made).

FR-7: Data Storage

- FR-7.1: The system shall store raw documents (paths to encrypted files), extracted text, embeddings (pgvector), risk scores, and explainability outputs in PostgreSQL.
- FR-7.2: The system shall keep embeddings for face and document vectors using pgvector.

FR-8: API & Frontend

- FR-8.1: The backend shall expose RESTful endpoints to the React frontend for uploading files, polling status, and retrieving results.
 - FR-8.2: The frontend shall present onboarding progress, real-time nudges, and final decision.
-

4. Non-Functional Requirements (NFR)

NFR-1: Security

- NFR-1.1: All PII stored at rest shall be encrypted with AES-256.
- NFR-1.2: All network communication shall use TLS 1.3.
- NFR-1.3: Role-based access must be enforced for reviewer and admin interfaces (RBAC is implied by Round 1; implement reviewers/admin separation).

NFR-2: Performance

- NFR-2.1: System shall process onboarding pipeline end-to-end in a time suitable for user interaction (target: minutes). *(Round 1 indicates days → minutes improvement.)*
- NFR-2.2: OCR and face verification modules shall operate concurrently (asynchronously) to minimize latency.

NFR-3: Scalability

- NFR-3.1: System design must be modular and container-ready to allow future horizontal scaling (Docker/Kubernetes).

NFR-4: Reliability & Availability

- NFR-4.1: The system shall persist intermediate results so failed tasks can be retried without total restart.
- NFR-4.2: Immutable audit logs must be durable and tamper evident.

NFR-5: Compliance & Explainability

- NFR-5.1: SHAP outputs and full feature maps must be available to compliance reviewers for each decision.
- NFR-5.2: All actions in the pipeline must be logged for AML/KYC compliance.

NFR-6: Maintainability

- NFR-6.1: Modules (OCR, FaceNet, Mediapipe, BERT, XGBoost, SHAP) must be independently deployable and upgradable.
-

5. Data Requirements & Schema Summary

(High-level; see data model document for details)

- **Users**(user_id UUID, name, contact, created_at, updated_at)
- **KYC_Applications**(application_id UUID, user_id FK, status, submitted_at, decision_at, risk_score_id FK)
- **Documents**(document_id UUID, application_id FK, document_type, raw_file_path (encrypted), extracted_text JSONB, ocr_confidence, created_at)
- **Embeddings**(embedding_id UUID, application_id FK, vector pgvector, embedding_type, created_at)
- **RiskScores**(risk_score_id UUID, application_id FK, score_value float, feature_map JSONB, created_at)
- **Explainability**(explain_id UUID, application_id FK, shap_summary JSONB, created_at)
- **AuditLogs**(log_id UUID, application_id FK, action_type, action_details JSONB, timestamp, actor)

Storage constraints:

- All file paths reference encrypted file storage.
 - Extracted text and SHAP summaries stored as JSONB.
-

6. System Interfaces

6.1 Frontend ↔ Backend

- REST API endpoints for:
 - Create application / start onboarding
 - Upload document / selfie
 - Poll processing status / retrieve results
 - Fetch explanation & audit entries

6.2 Backend ↔ Database

- PostgreSQL connections (secure, TLS).
- pgvector used for storing / querying embeddings.

6.3 Internal AI Services

- OCR, FaceNet, Mediapipe, BERT, XGBoost, SHAP are invoked by backend services (synchronous or asynchronous depending on orchestration).
-

7. User Interfaces

- Mobile/web friendly React UI for:
 - Document capture with guidance & nudges
 - Progress tracking
 - Display of final decision and, for reviewers, explainability view with SHAP summary and audit trace
-

8. Acceptance Criteria

- AC-1: A user can complete onboarding: upload docs & selfie, and receive a decision (approve / review) with evidence stored.
 - AC-2: OCR extracts key fields (Name, DOB, Address) and BERT standardizes them in the application record.
 - AC-3: FaceNet returns similarity embedding and Mediapipe returns liveness confidence; both are stored.
 - AC-4: XGBoost produces a risk score and SHAP outputs explaining the score are retrievable.
 - AC-5: Audit logs capture each pipeline step and are immutable.
 - AC-6: All PII is encrypted at rest and communications use TLS 1.3.
-

9. Constraints & Assumptions

- Constraints: Use specified components only; single environment hackathon deployment.
 - Assumptions: Users can capture legible documents and selfies; demo links will be public.
-

10. Risks & Mitigations

- Risk: Poor OCR on some document types → Mitigation: Dual OCR (EasyOCR primary + Tesseract fallback) and human-in-the-loop for edge cases.
 - Risk: False positives/negatives in face match or liveness → Mitigation: Store confidence scores, tune thresholds, route ambiguous cases to manual review.
 - Risk: Regulatory concerns about automated decisions → Mitigation: SHAP explainability + immutable audit logs + manual review path.
-

11. Traceability Matrix (example)

- FR-2 (OCR) → Data Model: Documents.extracted_text → NFR-1 (Security)
 - FR-3 (Face) → Data Model: Embeddings.vector → NFR-3 (Scalability)
 - FR-5 (Risk scoring) → Data Model: RiskScores.feature_map & Explainability.shap_summary → NFR-5 (Explainability)
-